

Week 0 - Applied - Practical



Objectives of this Prac

The objective is to become familiar with Python and, in particular, with its:

- List data structure.
- Iteration and selection.
- Input/output.



Useful Material

The following links provide useful info on:

- Iteration and Selection: <https://docs.python.org/3/tutorial/controlflow.html>.
- Lists and Tuples: <https://docs.python.org/3/tutorial/datastructures.html>.
- Style guide: <https://www.python.org/dev/peps/pep-0008/>.
- Docstrings: <https://www.python.org/dev/peps/pep-0257/>.
- Type hints: The beginning of <https://www.python.org/dev/peps/pep-0484/> has the basics.

Exercise 1

In this exercise you will practice reading a line from input that contains several values, separating it into strings, iterating over a list, and returning values.

1. Using PyCharm create a file **prac0.py** and add a Python function **read_integers** that asks the user to enter some integers separated by spaces in a single line, and returns a list of these integers in the order they were provided.
2. Run your code line by line and see how the value of your integers change. If you are confused about how to do so, please ask your demonstrator.
3. Copy your code for the function and place it in the Ed console and see if you pass the test cases.

The following shows a few examples of what should happen when you run the file in the interactive Ed console:

```
----- 1st run -----
Enter some integers:
[]
----- 2nd run -----
Enter some integers:-6
[-6]
----- 3rd run -----
Enter some integers: 1 -2 6 0
[1, -2, 6, 0]
```



You can use **strings = input("Some string:").split()** to display "**Some string**" and then read the input line, leaving in strings the list of strings in that line separated by spaces.



Please note - Don't worry about error checking your code right, pretend that the user is amazing and always passes the input in the right format

```
1 from typing import List
2 def read_integers(string_of_integers: str) -> List[int]:
3     from typing import List
4
5 def read_integers(string_of_integers: str) -> List[int]:
6     if string_of_integers == "":
7         return []
8
9     strings = string_of_integers.split()
10    numbers = []
11
12    for s in strings:
13        numbers.append(int(s))
14
15    return numbers
16
17
18 if __name__ == '__main__':
19     string_of_integers = input("Enter some integers:")
20     list_of_integers = read_integers(string_of_integers)
21     print(list_of_integers)
```

Description

Exercise 2

In this exercise you will practice taking arguments, a bit of maths, and more iteration and returning elements.

Add a Python function **sum_squared_integers** to the **prac0.py** file that, given a list of integers, returns the sum of squaring each of the integers in the list. The following shows a few examples:

```
>>> from prac0 import sum_squared_integers
>>> sum_squared_integers ([])
0
>>> sum_squared_integers([ -6])
36
>>> sum_squared_integers([1, -2, 6, 0])
41
>>>
```



 prac0.py

```
1 def sum_squared_integers(numbers):
2     total = 0
3
4     for n in numbers:
5         total += n * n
6
7     return total
```

Exercise 3

In this exercise you will practice reading from a file, rather than from input.

Add a Python function **read_from_file_sum_squares** to the **prac0.py** file that, instead of asking users to input the integers, asks them to give the name of a file, where each line in the file has a sequence of integers separated by spaces. For each line in the file, the function should read the integers, compute the sum of their squares and print out the result. If the line is empty, it should print a 0. We have provided 2 test files (**file1** and **file2**) for testing.

The following shows the result of calling function **read_from_file_sum_squares()** for each of these two files:

```
>>> from prac0 import read_from_file_sum_squares
>>> read_from_file_sum_squares()
Enter the filename:file1
9
>>> read_from_file_sum_squares()
Enter the filename:file2
14
20
0
1
>>>
```



You can use **input_file = open(filename, "r")** to open the file named by string **filename** to read, and the line **list_lines = input_file.readlines()** to store in **list_lines** the list of strings, where each string is a line of the file, in the order they appear in the file. Remember to close the file once you do not need it anymore, using **input_file.close()**.

The image shows a code editor window with a sidebar on the left and a main editing area on the right. The sidebar contains a 'Files' panel with a '+' icon and three files: 'file1' (selected), 'file2', and 'prac0.py'. The main area displays the code for 'prac0.py'.

```
1 3def read_from_file_sum_squares():
2     filename = input("Enter the filename:")
3     input_file = open(filename, "r")
4     list_lines = input_file.readlines()
5
6     for line in list_lines:
7         numbers = line.split()
8         total = 0
9
10        for n in numbers:
11            total += int(n) * int(n)
12
13        print(total)
14
15    input_file.close()
```

Files

+

file1

file2

prac0.py

file2

1 1 2 -3

2 4 2

3

4 1

prac0.py

file1

>_

[]

⚙

Files

+

file1

file2

prac0.py

file2

1

prac0.py

file1

>_

[]

⚙

Description

Exercise 4

In this exercise you will practice nested iterations and building lists of lists.

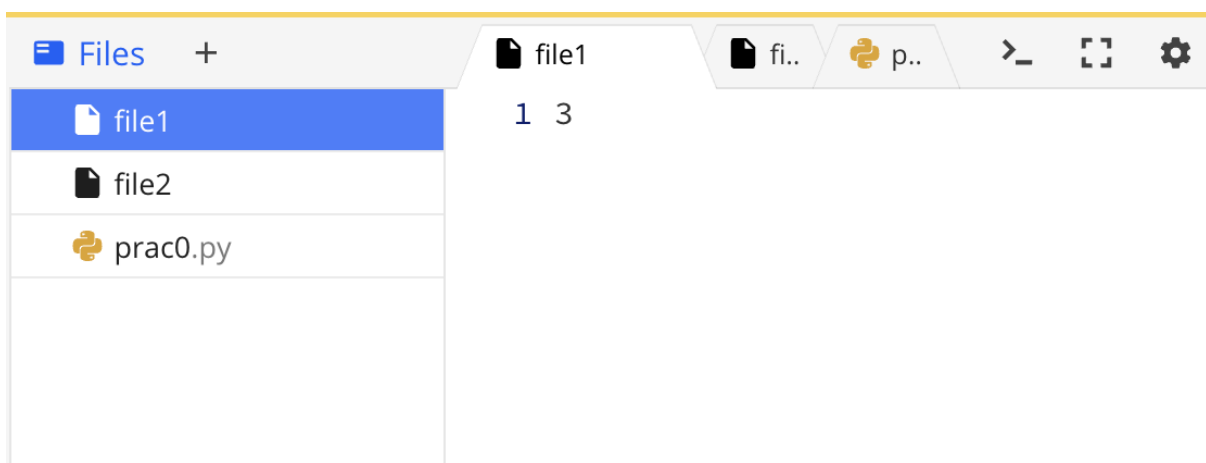
Add a Python function **read_from_file_table()** to the **prac0.py** file that reads a file as before, and stores every non-negative number in each line in a list, in the order provided, obtaining a table (i.e., a list of lists). If a line is empty, it should add the empty list as an element of the table. For the two file above, the result is as follows:

```
>>> from prac0 import read_from_file_table
>>> read_from_file_table()
Enter the filename:file1
[[3]]
>>> read_from_file_sumtable()
Enter the filename:file2
[[1, 2], [4, 2], [], [1]]
>>>
```



Note that you can import several functions in one line (separated by commas), for example:

```
>>> from prac0 import read_integers , sum_squared_integers
```



Files +

file1file2prac0.py

>_[]⚙

file1

file2

prac0.py

```
1 def read_from_file_table():
2     filename = input("Enter the filename:")
3     input_file = open(filename, "r")
4     list_lines = input_file.readlines()
5
6     table = []
7
8     for line in list_lines:
9         numbers = line.split()
10        row = []
11
12        for n in numbers:
13            value = int(n)
14            if value >= 0:
15                row.append(value)
16
17        table.append(row)
18
19    input_file.close()
20    return table
```

Files +

file1file2prac0.py

>_[]⚙

file1

file2

prac0.py

```
1
```

Examples of Good Time Complexity Analysis

In FIT1008, we usually ask you to include the time complexity analysis of the functions you write in the function's documentation.

Here are some important points to have in mind when you're doing so:

- Make sure you are defining your variables properly. Your function may have some arguments. You should clearly state what your analysis variables are and how they relate to the function inputs.
- If the function has different time complexities in the best case and worst case scenarios, make sure you separate these, and you explain when those cases happen.
- For the sake of your assignments and marking, make sure you provide a brief explanation for your analysis. The goal is not to explain everything line by line, it's only to demonstrate you understand the analysis and haven't put random numbers there.

Here are some examples of invalid ways of providing time complexity in your functions, based on the assignment submissions we have seen over the last few years:

❌ Invalid Analysis 1

```
def my_sort_function(my_list):  
    for i in range(len(my_list)): # This runs N times  
        at_least_one_swap = False  
        for j in range(i + 1, len(my_list)): # This runs N - i times  
            if my_list[i] > my_list[j]: # These are constant  
                my_list[i], my_list[j] = my_list[j], my_list[i]  
                at_least_one_swap = True  
        if not at_least_one_swap:  
            break  
    # The whole thing is N^2
```

- Analysis should be placed in the docstrings of the function at the top
- Analysis should define the variables
- No need to provide line by line explanation, include them all at the top

❌ Invalid Analysis 2

```
def my_sort_function(my_list):  
    """  
    :complexity: O(N^2) where N is the length of my_list.  
    It's quadratic because there are 2 loops.  
    """  
    for i in range(len(my_list)):  
        at_least_one_swap = False  
        for j in range(i + 1, len(my_list)):  
            if my_list[i] > my_list[j]:  
                my_list[i], my_list[j] = my_list[j], my_list[i]  
                at_least_one_swap = True  
        if not at_least_one_swap:  
            break
```

- Good variable definition, but is it always N^2 ? Or is it different in different cases?
- It is quadratic, but is it only because there are 2 for loops? Many functions can have 2 for loops without being quadratic. So your marker here can't give full marks because you are not demonstrating that you understand your analysis.

❌ Invalid Analysis 3

```
def my_sort_function(my_list):  
    """  
    :complexity: Best case is  $O(1)$  when the input list only has a few elements so  
    there wouldn't be more than a constant number of operations.  
  
    Worst case is  $O(N^2)$  where  $N$  is the length of my_list. It's  $N^2$  because  
    the inner loop and its statements run  $n-1$  times for  $i = 1$ ,  $n-2$  times for  $i = 2$ ,  $n-3$  times  
    for  $i = 3$ , ..., until 1 time for  $i = n - 1$ . So in total it would run  $1 + 2 + \dots + n-1$  times  
    which is  $O(N^2)$ .  
    """  
    for i in range(len(my_list)):  
        at_least_one_swap = False  
        for j in range(i + 1, len(my_list)):  
            if my_list[i] > my_list[j]:  
                my_list[i], my_list[j] = my_list[j], my_list[i]  
                at_least_one_swap = True  
        if not at_least_one_swap:  
            break
```

- We don't analyse time complexity when the input size is limited. And we certainly don't put the limit on the input size to create the best case scenario. Remember, the main purpose of time complexity is to inform us how the function will **scale** as the input size grows. It is true that with an empty list, the function won't run that many operations, but that's not very informative and it's not what we need to analyse.

✅ Valid Analysis

```
def my_sort_function(my_list):  
    """  
    :complexity: Best case is  $O(N)$  where  $N$  is the length of my_list.  
    Best case happens when the list is already sorted. In which case, the inner loop goes through  
    and no elements are swapped so it hits an early termination.  
  
    Worst case is  $O(N^2)$  where  $N$  is the length of my_list. It's  $N^2$  because  
    the inner loop and its statements run  $n-1$  times for  $i = 1$ ,  $n-2$  times for  $i = 2$ ,  $n-3$  times  
    for  $i = 3$ , ..., until 1 time for  $i = n - 1$ . So in total it would run  $1 + 2 + \dots + n-1$  times  
    which is  $O(N^2)$ .  
    """  
    for i in range(len(my_list)):  
        at_least_one_swap = False  
        for j in range(i + 1, len(my_list)):  
            if my_list[i] > my_list[j]:  
                my_list[i], my_list[j] = my_list[j], my_list[i]  
                at_least_one_swap = True  
        if not at_least_one_swap:  
            break
```

- Variables are defined properly
- Best case and worst case scenarios are separated
- We explain **when** the two different cases would happen
- We explain **why** the complexity would be that in each of the cases
- We provide all of this in the function documentation, while keeping it short and succinct.

Appendix

Some things to note about the code snippets:

- Function **input** is what's called a "built-in" function. It's part of Python proper and available for all and every program. There are other ways for you to acquire functions (you may import them from the Python standard library, you may install additional packages, or you may write them yourself).
- Calls to **input()** are usually followed by a call to string method **strip()**.
- Note also how we use **not** (rather than **!**) for negating the boolean value of an expression in, for instance, a conditional.
- The call to **int(x)** function converts the number or string **x** to an integer. It is also a builtin, and one of the many available for type conversion.